# React Native and TypeScript Cheat Sheet

*High yield syntax, reasoning patterns, and common traps*

Independently prepared study materials based on the ten supplied Autumn 2026 lecture decks. Sources checked 6 November 2026. The practice questions and rubrics are original and are not official assessments.

Use this sheet for study after trying to recall a pattern yourself. It contains code and is not automatically an allowed assessment aid. Examples are fragments unless explicitly labeled as complete files.

## Arrow bodies

An expression body returns automatically. A block body needs return to provide a value.

```
const twice = (n: number) => n * 2;
const twiceBlock = (n: number) => { return n * 2; };
const make = (name: string) => ({ name, popular: false });
```

Common trap: A newline or parentheses does not turn a block into an expression. A bare object-looking brace after => begins a block.

## Map into elements

map transforms data into returned values. JSX can render an array of elements.

```
{items.map(item => (
  <ItemCard key={item.id} item={item} />
))}
```

Common trap: Put key on the outermost returned sibling. Missing return produces undefined values; random keys are unstable.

## State inference

Let useful initial values provide types; annotate empty or nullable state.

```
const [name, setName] = useState('');
const [items, setItems] = useState<Item[]>([]);
const [selected, setSelected] = useState<Item | null>(null);
```

Common trap: A typed initial array supports inference. An empty literal or null does not describe all future values.

## Two valid setter forms

Pass the next value, or a pure function that computes the next value from pending state.

```
setCount(0);
setCount(previous => previous + 1);
setItems(previous => [...previous, item]);
```

Common trap: Setters accept updater functions. Do not mutate previous or perform side effects inside an updater.

## Immutable array operations

Use a new array; copy the changed object when editing an item.

```
setItems(prev => [...prev, item]);
setItems(prev => prev.filter(x => x.id !== id));
setItems(prev => prev.map(x =>
  x.id === id ? { ...x, popular: !x.popular } : x
));
```

Common trap: Spread is shallow. [...prev] followed by changing a nested object still mutates shared data.

## Event function versus call

Pass a function to an event prop. Use a wrapper when choosing arguments.

```
<Button title="Save" onPress={save} />
<Pressable onPress={() => removeItem(item.id)} />
```

Common trap: onPress={save()} invokes save during render. It usually passes the wrong value and may cause repeated updates.

## Callback props

The parent owns shared state and sends a callback contract to the child. This callback accepts a draft; the parent supplies a stable ID when it creates the stored item.

```
type Draft = { name: string; popular: boolean };
type Props = { onAdd: (draft: Draft) => void };
// Parent JSX: <Composer onAdd={addItem} />
// Child event: onAdd({ name, popular });
```

Common trap: The child must construct the argument shape the callback expects. Props travel down; the callback reports an action to the owner.

## Controlled text input

The displayed value and change callback must refer to the same state.

```
<TextInput value={name} onChangeText={setName} />
const cleanName = name.trim();
// In submit handler: if (!cleanName) return;
```

Common trap: onChangeText gives a string, not a browser event. A fixed value without an update makes edits appear stuck.

## JSX and conditions

Use expressions inside JSX braces and put visible strings in Text.

```
{count > 0 ? <Text>{count} items</Text> : null}
<Text style={{ fontSize: 18 }}>Hello</Text>
```

Common trap: 0 && <Text>…</Text> evaluates to 0. Double braces in style={{…}} are a JSX expression containing an object.

## Flexbox axes

The parent controls direct children. React Native defaults to column.

```
row: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  alignItems: 'center',
}
```

Common trap: justifyContent uses the main axis; alignItems uses the cross axis. With column, the main axis is vertical.

## Box model and dimensions

Padding is inside the border; margin is outside. Numeric sizes are logical layout units.

```
card: {
  margin: 16, padding: 16,
  borderWidth: 1, borderColor: '#667085',
  borderRadius: 12,
}
```

Common trap: Use camelCase style names. A percentage or flex size needs a parent with usable dimensions; avoid assuming flex: 1 means full screen in every context.

## Import and export pairs

A module has at most one default export and can have several named exports.

```
// Card.tsx: export default Card; export const color = 'navy';
import Card, { color } from './Card';
import type { Item } from './types';
```

Common trap: Default imports do not use braces. Named imports match the export or use an explicit as alias.

## Effect and cleanup

Use an effect to synchronize with an external system and clean up resources.

```
useEffect(() => {
  if (!running) return;
  const id = setInterval(() => setSeconds(s => s + 1), 1000);
  return () => clearInterval(id);
}, [running]);
```

Common trap: Do not call hooks conditionally. [] is mount-oriented, but development Strict Mode may run an extra setup-cleanup cycle.

## Promise values and order

then attaches a reaction and returns a Promise. Return a value to pass it onward.

```
console.log('A');
Promise.resolve(3)
  .then(n => n * 2)
  .then(n => console.log(n));
console.log('B'); // A, B, then 6
```

Common trap: A missing return in a block causes the next stage to receive undefined. Promise callbacks run after current synchronous work.

## Debug in a useful order

Read the exact error, identify the smallest failing expression, and predict a test before editing.

```
// Ask: What value is expected here?
// Is this an array [], object {}, function (), or JSX element?
// Is this callback being passed, called, or returning a value?
```

Common trap: Do not fix several unrelated things at once. A small compiling fragment is easier to extend than a large unfinished component.

**Read more:** React useState / React Rendering Lists / React Updating Arrays in State / React Sharing State Between Components / React useEffect / TypeScript Everyday Types / React Native Layout with Flexbox